

SmartSafe Master Unit

Complete Design and Code Explanation of the 4 x 4 Matrix Keypad Module

Microprocessors End-Term Project

July 2026

Abstract

This report documents the 4 x 4 matrix-keypad subsystem used by the SmartSafe Master. It explains the physical row-column switch matrix, the mixed-port pin assignment chosen to avoid conflicts with SPI, USART, Timer1, ADC, and the analog comparator, the ATmega32 data-direction, output, and input registers, the reason columns use internal pull-up resistors, the reason one row is driven low at a time, the complete scanning sequence, the relationship between physical coordinates and the software key map, input settling time, edge-style key reporting, limitations of the present debounce method, ghosting and simultaneous-key behavior, and integration with the password-entry state machine. The module does not require a dedicated timer or communication baud rate; those absences are also explained rather than hidden.

Contents

1 Purpose of the Keypad	3
2 How a 4 x 4 Matrix Keypad Is Constructed	3
3 Final SmartSafe Pin Assignment	3
4 Physical Key Layout and Key Map	4
5 ATmega32 GPIO Register Model	4
6 Why Columns Use Internal Pull-Ups	5
7 Why One Row Is Pulled LOW at a Time	5
8 GPIO Abstraction Used by the Driver	5
9 Complete Scanning Sequence	6
10 Settling Delay Calculation	6
11 Key Detection Logic	7
12 One Report per Press	7
13 Debounce: What the Present Code Does and Does Not Do	7
14 Why the Keypad Module Has No Baud Rate	7
15 Integration with the Main Loop	8
16 Simultaneous Keys, Ghosting, and Output Contention	8

17 Register and Constant Summary	8
18 Proteus Test Procedure	9
19 Common Failure Modes	9
20 Conclusion	9

1 Purpose of the Keypad

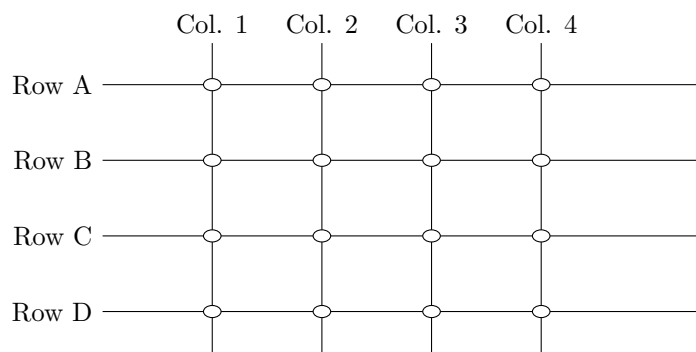
The keypad is the local user-input device of the SmartSafe Master. It provides ten decimal digits and several function keys. The application accepts only:

- digits 0 through 9, which are stored in a four-character password buffer;
- C, which clears the current partial password.

After four digits are entered, the main loop automatically checks the password. The E, arithmetic, decimal-point, and division keys are ignored by the password logic.

2 How a 4 x 4 Matrix Keypad Is Constructed

A 4 x 4 keypad contains 16 normally open switches arranged at the intersections of four row conductors and four column conductors. Pressing a key electrically connects exactly one row to exactly one column.



Each intersection is one normally open key switch

Figure 1: Electrical organization of the matrix keypad.

Only eight microcontroller pins are required for sixteen keys. A separate input pin for every key would require sixteen pins.

3 Final SmartSafe Pin Assignment

The final mapping avoids all peripheral conflicts:

Keypad terminal	ATmega32 pin	Direction during scanning
A	PA4	Output row 0
B	PA5	Output row 1
C	PA6	Output row 2
D	PA7	Output row 3
1	PD3	Input column 0 with pull-up
2	PD4	Input column 1 with pull-up
3	PB0	Input column 2 with pull-up
4	PB1	Input column 3 with pull-up

This mixed-port arrangement is deliberate:

- PB4–PB7 are reserved for the hardware SPI EEPROM;

- PB3 is the AIN1 power-monitor input;
- PD0 and PD1 are USART RXD and TXD;
- PD2 is the INT0 wake button;
- PD5 is OC1A for the servo;
- PA3 is ADC3 for the tamper potentiometer;
- PA0 and PA2 control the LCD.

4 Physical Key Layout and Key Map

For the final Proteus calculator keypad, the intended mapping is:

	Column 1	Column 2	Column 3	Column 4
Row A	7	8	9	/
Row B	4	5	6	*
Row C	1	2	3	-
Row D	C	0	E	+

The matching C array is:

```

1 static const char keypad[4][4] = {
2     { '7', '8', '9', '/' },
3     { '4', '5', '6', '*' },
4     { '1', '2', '3', '-' },
5     { 'C', '0', 'E', '+' }
6 };

```

Listing 1: Key map matching the final Proteus component layout.

The key map is not an arbitrary software choice. Its row and column indices must correspond to the actual labels and terminal order of the selected Proteus component.

5 ATmega32 GPIO Register Model

Every port uses three important registers:

Register	When written/read	Meaning
DDRx	Written	Select input (0) or output (1) for each pin
PORTx	Written	Output level for output pins; pull-up control for input pins
PINx	Read	Actual logic level currently present at the pin

For a column input with an internal pull-up:

```

1 DDRx  &= ~(1U << bit);  /* input */
2 PORTx |= (1U << bit);  /* internal pull-up enabled */

```

For an output row driven HIGH:

```

1 DDRx  |= (1U << bit);
2 PORTx |= (1U << bit);

```

For the selected output row driven LOW:

```

1 DDRx  |=  (1U << bit);
2 PORTx &= ~(1U << bit);

```

6 Why Columns Use Internal Pull-Ups

When no key is pressed, a keypad column is otherwise disconnected. A disconnected CMOS input is floating and may read either HIGH or LOW because of noise, leakage, or simulation state. The internal pull-up resistor gives every idle column a defined HIGH state without four external resistors.

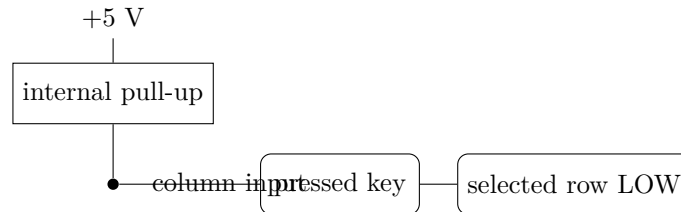


Figure 2: A pressed key overcomes the weak pull-up and connects the selected column to the LOW row.

With no key pressed, the column reads HIGH. With a key pressed on the currently selected row, it reads LOW.

7 Why One Row Is Pulled LOW at a Time

The selected row must drive the opposite level from the column idle bias. Since the columns idle HIGH through pull-ups, the active row is LOW. Pressing a key then changes the corresponding column from HIGH to LOW.

Rows are selected one at a time because the software must identify both coordinates. If every row were LOW simultaneously, a LOW column would identify the column but not which of the four keys in that column was pressed.

The reverse electrical method is possible: columns could use external pull-down resistors and one row could be driven HIGH. The ATmega32 provides internal pull-ups but no internal pull-downs, so active-LOW row scanning uses fewer components.

8 GPIO Abstraction Used by the Driver

Because the rows and columns are spread across PORTA, PORTB, and PORTD, the driver stores register addresses and bit numbers in a small structure:

```

1 typedef struct {
2     volatile uint8_t *ddr;
3     volatile uint8_t *port;
4     volatile uint8_t *pin;
5     uint8_t bit;
6 } gpio_t;

```

Arrays then describe the eight keypad terminals. The same helper routines can configure a pin regardless of which AVR port owns it. This is cleaner than duplicating separate register expressions for each row and column.

9 Complete Scanning Sequence

A non-blocking scan follows this sequence:

1. configure all four rows as HIGH outputs;
2. configure all four columns as pull-up inputs;
3. for row 0 through row 3:
 - a. return every row HIGH;
 - b. drive the current row LOW;
 - c. wait briefly for the electrical levels to settle;
 - d. read all four column inputs;
 - e. when a column is LOW, convert the row-column pair through the key map;
4. return every row HIGH;
5. report a newly pressed key once.

```

1  for (uint8_t r = 0; r < 4U; r++) {
2      for (uint8_t rr = 0; rr < 4U; rr++)
3          gpio_output_high(&rows[rr]);
4
5      gpio_output_low(&rows[r]);
6      __asm__ volatile ("nop\n\tnop\n\tnop\n\tnop\n\t");
7
8      for (uint8_t c = 0; c < 4U; c++) {
9          if (gpio_is_low(&cols[c])) {
10             pressed = keymap[r][c];
11             break;
12         }
13     }
14
15     if (pressed)
16         break;
17 }

```

Listing 2: Core matrix scan.

10 Settling Delay Calculation

The driver uses four NOP instructions after changing the active row. Each NOP consumes one CPU clock cycle. At 8 MHz:

$$T_{CPU} = \frac{1}{8\,000\,000} = 0.125 \mu s.$$

Four instructions therefore provide approximately

$$4 \times 0.125 = 0.5 \mu s$$

of explicit delay, in addition to the execution time of surrounding instructions. A keypad matrix has very small capacitance and low operating speed, so this is adequate for the Proteus simulation and ordinary short wiring.

No timer register is required for this settling delay. Using a hardware timer for a fraction of a microsecond would add configuration overhead larger than the intended delay.

11 Key Detection Logic

The helper reads the actual input register:

```
1 static uint8_t gpio_is_low(const gpio_t *g)
2 {
3     return ((*g->pin) & (1U << g->bit)) == 0U;
4 }
```

The driver must read PINx, not PORTx. PORTx holds the output latch or pull-up setting; it does not necessarily represent the external electrical level.

12 One Report per Press

The driver uses a static state variable:

```
1 static char last_reported = 0;
```

The behavior is:

- no key currently detected: clear `last_reported`;
- same key remains held: return zero;
- different or newly pressed key: return that character and remember it.

This prevents one long press from filling all four password positions in successive main-loop iterations.

13 Debounce: What the Present Code Does and Does Not Do

The edge-style state suppresses repeated reports while the key remains continuously detected. It is not a complete time-based debounce algorithm. A mechanical contact can briefly alternate between pressed and released during the first several milliseconds. If one scan sees no key and the next scan sees the key again, `last_reported` may be cleared and the same physical press could theoretically be reported twice.

Proteus keys are usually clean enough for the current approach. A physical design can add a timer-based confirmation, for example requiring the same row-column result for 20 ms before acceptance. The existing Timer2 compare interval is approximately 9.984 ms, so two or three stable Timer2 intervals can provide approximately 20–30 ms of debounce without a blocking delay.

14 Why the Keypad Module Has No Baud Rate

Baud rate is a serial-communication concept. The keypad is directly connected to GPIO pins and does not transmit asynchronous serial frames. Therefore:

- no USART register is configured by the keypad driver;
- no baud-rate calculation applies to the keypad itself;
- the CPU clock matters only for scan execution time and the short NOP settling interval.

The Master later uses USART to report the result of password processing, but that communication belongs to the application and USART modules rather than the keypad electrical interface.

15 Integration with the Main Loop

The main loop calls:

```
1 char key = keypad_scan();
2 if (!key)
3     continue;
```

A valid key resets the inactivity counter. The application then handles it as follows:

- C: clear the buffer and redraw the prompt;
- 0–9: display an asterisk and append the digit;
- every other key: ignore it;
- four digits: compare the buffer with the password stored in internal EEPROM.

The keypad is intentionally ignored during lockout, while the servo is open, and during event-related UI holds. This policy is enforced by the main loop rather than the low-level keypad driver.

16 Simultaneous Keys, Ghosting, and Output Contention

A simple matrix without a diode at every switch is designed primarily for one key at a time. Multiple keys can create ambiguous current paths known as ghosting. Some combinations can also connect a HIGH nonselected row to the LOW selected row through two closed keys.

For a four-digit safe password, the user is expected to press and release one key at a time, so the simple matrix is adequate. A production keyboard requiring reliable multi-key operation would use per-key diodes or a dedicated keypad controller.

17 Register and Constant Summary

Item	Selection	Reason
DDRA4–7 PORTA4–7	outputs one LOW at a time	Four keypad rows. Select one active row against HIGH-biased columns.
DDRD3, DDRD4, DDRB0, DDRB1	inputs	Four keypad columns.
Corresponding PORT bits	1 while inputs	Enable internal pull-ups.
Corresponding PIN bits	read	Observe actual column levels.
F_CPU	8 MHz	System clock; gives 0.125 microsecond per CPU cycle.
NOP count	4	About 0.5 microsecond explicit settling time.
Rows/columns	4 and 4	Sixteen keys using eight MCU pins.
Dedicated timer	none	Scanning is quick and non-blocking; no periodic timer is required.
Baud rate	not applicable	The keypad is a direct GPIO matrix, not a serial device.

18 Proteus Test Procedure

1. Set the ATmega32 clock property to 8 MHz.
2. Verify terminals A–D connect to PA4–PA7.
3. Verify terminals 1–4 connect to PD3, PD4, PB0, and PB1.
4. Start the simulation and confirm the Master LCD shows `Enter Password:.`
5. Press one digit. Exactly one asterisk should appear.
6. Hold the key. Additional asterisks should not appear.
7. Release and press the same key again. A second asterisk should appear.
8. Press C; all partial input should be cleared.
9. Enter four digits and verify automatic password processing.

19 Common Failure Modes

Symptom	Likely cause and correction
Every key returns the wrong symbol	Key-map rows or columns do not match the Proteus terminal order.
No key is detected	Rows/columns are wired incorrectly, column pull-ups are disabled, or the Master is in sleep/lockout/hold state.
One press enters several digits	Edge state was removed or switch bounce requires timed debounce.
A column always reads LOW	Short circuit, a key is held, or the selected pin direction is wrong.
SPI EEPROM stops working	Keypad was connected to PB4–PB7, conflicting with hardware SPI.
C key does not clear input	Physical C key is at a different row-column location than the software key map.

20 Conclusion

The keypad module converts a 16-key switch matrix into a unique row-column coordinate by driving one row LOW at a time and reading four pull-up columns. The design uses the ATmega32 internal pull-ups, so no external column resistors are required. Mixed-port GPIO descriptors avoid conflicts with the project peripherals, four NOP instructions provide a short settling interval, and the static last-key state prevents repeated entry while a key is held. No USART baud rate or dedicated timer is required because the keypad is a directly scanned GPIO device.

References

- [1] Microchip Technology Inc., *ATmega32A Data Sheet*, sections on I/O ports, internal pull-ups, and instruction timing.
[Official Microchip PDF](#)
- [2] Microprocessors End-Term Project, *Smart Safe with Remote Alarm*, project specification supplied with the implementation.

[3] SmartSafe source files: `keypad.c`, `keypad.h`, `config.h`, and `main.c`, July 2026 project version.